

Proyecto Final de Diseño y Análisis de Algoritmos

Problema: “El Comerciante Holandés”

Claudia Hernández Pérez Joel Aparicio Tamayo

30 de diciembre de 2025

Resumen

Este documento presenta el análisis completo del problema “*El Comerciante Holandés*”. Se incluye la formalización matemática del problema, análisis de complejidad computacional, diseño de algoritmos por fuerza bruta y eficiente (con uso de metaheurística), y finalmente una comparación entre ambos enfoques.

1. Definición del problema

La prestigiosa Compañía Holandesa de las Indias Orientales, en su afán por dominar el comercio mundial, se enfrenta a un desafío monumental. Un capitán experimentado, al mando de una de sus valiosas flotas, debe emprender una expedición comercial que partirá de Ámsterdam y, tras recorrer los puertos más lucrativos del Viejo y Nuevo Continente, deberá regresar a su puerto de origen.

Los inversores de la Compañía han proporcionado un capital inicial considerable y han establecido un plazo máximo para la duración de la expedición. El capitán tiene la libertad de elegir qué puertos visitar y en qué orden, con la única condición de no visitar el mismo puerto dos veces en el mismo viaje (por cuestiones de acuerdos comerciales y evitar saturación del mercado).

En cada puerto, el capitán encontrará una lista de mercancías disponibles, con sus respectivos precios de compra y venta (que pueden variar significativamente). El capitán puede vender las mercancías que lleva a bordo y comprar nuevas. Sin embargo, debe ser astuto:

- La capacidad de carga de su barco es limitada, por lo que no puede llevar más de lo que su bodega permite.
- No es necesario vender todas las mercancías al llegar a un puerto; el capitán puede decidir retener parte de su cargamento si cree que podrá venderlo a un precio más alto en un puerto posterior.
- Debe asegurarse de que, después de cada operación de compra, le quede suficiente dinero para cubrir los salarios de la tripulación, los impuestos portuarios y las posibles reparaciones del barco hasta el siguiente destino.
- El tiempo es oro; la duración total del viaje, incluyendo el tiempo de navegación entre puertos, no debe exceder el plazo fijado por los inversores.

El objetivo del capitán es claro: planificar la ruta y las transacciones en cada puerto de tal manera que, al regresar a Amsterdam, el capital final de la expedición sea el máximo posible, superando con creces la inversión inicial.

2. Formalización del Problema

Antes de abordar cualquier solución algorítmica, es fundamental establecer una representación matemática precisa del problema. Esta sección presenta nuestra formalización del Comerciante Holandés, definiendo las estructuras de datos de entrada, las restricciones que deben cumplirse y la función objetivo a optimizar. Este modelo elimina las ambigüedades de la descripción narrativa original.

2.1. Definiciones Matemáticas

Sea:

- $G = (V, E)$: grafo completo de n puertos, donde $V = \{0, 1, \dots, n - 1\}$ (0 representa Ámsterdam, punto de inicio y fin).
- $d_{i,j}$: tiempo de navegación entre puertos i y j . Se asume $d_{i,i} = 0$, $\forall i$ (no se navega a ningún lugar).
- T_{max} : tiempo máximo total permitido para la expedición.
- C_{max} : capacidad máxima de carga del barco.

- k_0 : capital inicial proporcionado por los inversores.
- $k_{\min}$: capital mínimo que debe mantenerse tras cada transacción para el pago de impuestos portuarios (son los mismos en todos los puertos) y salarios de la tripulación, que incluyen un bonus extra para que costeen las posibles reparaciones.

En cada puerto $i \in V$:

- $M_i = [(w_{i,k}, p_{i,k}^{compra}, p_{i,k}^{venta})]_{k=1}^{m_i}$: lista de m_i mercancías disponibles, donde:
 - $w_{i,k} \in \mathbb{R}$: peso de la mercancía k en puerto i
 - $p_{i,k}^{compra} \in \mathbb{R}$: precio de compra de la mercancía k en puerto i
 - $p_{i,k}^{venta} \in \mathbb{R}$: precio de venta de la mercancía k en puerto i
- **Importante:** No hay garantía de que $p_{i,k}^{venta} > p_{j,k}^{compra} \forall i \neq j$. Pueden haber mercancías con pérdida potencial, lo que obliga al capitán a tomar decisiones estratégicas. Sin embargo sí se garantiza que $p_{i,k}^{venta} \leq p_{i,k}^{compra} \forall i$ (los del puerto tampoco son tontos, no van a pagar más de lo que ofertan).

2.2. Estado del Sistema

El estado al llegar a un puerto i se define como una tupla:

$$S_i = (K_i, L_i, R_i, t_i)$$

donde:

- $K_i \in \mathbb{R}$: capital disponible al llegar al puerto i .
- $L_i \subseteq \{m = (i', k, w, p^{compra}) : i', k \in \mathbb{N}; w, p^{compra} \in \mathbb{R}\}$: conjunto de mercancías a bordo, donde cada mercancía está identificada por:
 - i' : puerto donde fue comprada
 - k : índice de la mercancía
 - w : peso de la mercancía
 - p^{compra} : precio pagado por la mercancía
- $R_i \subseteq V$: conjunto de puertos ya visitados (ruta parcial).
- $t_i \in \mathbb{R}^+ \cup \{0\}$: tiempo acumulado hasta llegar al puerto i .

Para los estados inicial y final (ambos con $i = 0$) se definen las tuplas: $S_{inicial} = (k_0, [], [0], 0)$ y $S_{final} = (K_{final}, L_f, R_f, t_f)$

2.3. Operaciones

Al llegar a un puerto i , el capitán puede realizar:

1. **Venta parcial:** Para cada mercancía $G = (i_g, k, w_g, p_g^{compra}) \in L_i$, el capitán decide si venderla en el puerto actual i obteniendo $p_{i,k}^{venta}$ (precio de venta de esa mercancía en puerto i). Sea $M'_i \subseteq L_i$ el subconjunto de mercancías vendidas:

$$K_i \leftarrow K_i + \sum_{g \in M'_i} p_{i,k}^{venta}$$

Las mercancías vendidas se eliminan de la carga: $L_i \leftarrow L_i \setminus M'_i$.

2. **Compra selectiva:** Considerando las mercancías disponibles en M_i , el capitán selecciona un subconjunto $C_i \subseteq M_i$ para comprar, sujeto a:

- **Restricción financiera:**

$$K_i - \sum_{(w, p^{compra}, p^{venta}) \in C_i} p^{compra} \geq k_{\min}$$

- **Restricción de capacidad:**

$$\sum_{g \in L_i} w_g + \sum_{(w, p^{compra}, p^{venta}) \in C_i} w \leq C_{max}$$

Las mercancías compradas se añaden a bordo:

$$\forall m_k = (w, p^{compra}, p^{venta}) \in C_i : \quad L_i \leftarrow (i, k, w, p^{compra})$$

Luego se actualiza el capital:

$$K_i \leftarrow K_i - \sum_{(w, p^{compra}, p^{venta}) \in C_i} p^{compra}$$

Cuando termine sus transacciones debe **seleccionar su próximo destino**:

Sea $j \in V \setminus (R_i \cup \{i\})$ un candidato a próximo puerto, j es seleccionable si:

$$t_i + d_{i,j} \leq T_{max}$$

Si ningún j cumple esta condición, el camino es inválido. Luego de seleccionar el próximo destino, se actualiza el recorrido actual:

$$K_j \leftarrow K_i - k_{\min}$$

$$\begin{aligned}
L_j &\leftarrow L_i \\
R_j &\leftarrow R_i \cup \{i\} \\
t_j &\leftarrow t_i + d_{i,j} \\
i &\leftarrow j
\end{aligned}$$

2.4. Función Objetivo

$$\text{máx } K_f \quad ,$$

donde K_f es el capital tras vender cualquier carga restante en Ámsterdam (a precios de Ámsterdam).

sujeto a:

1. **Ruta simple:** Sea $\pi = (v_0, v_1, \dots, v_m, v_{m+1})$ la secuencia de puertos visitados, donde $v_0 = v_{m+1} = 0$ (Ámsterdam). Entonces:

$$\forall i \in \{1, \dots, m\} : v_i \in V \setminus \{0\} \quad \wedge \quad \forall i \neq j \in \{1, \dots, m\} : v_i \neq v_j$$

Es decir, ningún puerto no-inicial se repite en la ruta.

2. **Tiempo límite:** $t_f \leq T_{max}$, siendo t_f el tiempo de llegada a Ámsterdam al finalizar el viaje.
3. **Capital suficiente para operatividad:** $\forall i, K_i \geq k_{\min}$.
4. **Capacidad:** $\forall i, \sum_{G=(i_g, k, w_g, p_g^{compra}) \in L_i} w_g \leq C_{max}$.

3. Análisis de Complejidad Computacional

El problema es presentado originalmente como un problema de optimización, cuyo objetivo es hallar una ruta y una estrategia de transacciones que maximicen el capital final K_f . Para demostrar su clasificación en la jerarquía de complejidad, se define su versión de decisión asociada:

Dada la instancia del problema descrita en la Sección 2 y un umbral de capital $K_{objetivo} \in \mathbb{R}$, ¿existe una secuencia de puertos y transacciones tales que el tiempo total no exceda T_{max} y el capital al regresar a Ámsterdam sea al menos $K_{objetivo}$?

A continuación, se demuestra que este problema es **NP-Completo**.

3.1. Pertenencia a la Clase NP

Sea un certificado de solución una secuencia de puertos $\pi = (v_0, v_1, \dots, v_m, v_{m+1})$ y una secuencia de conjuntos de transacciones (compras C_i y ventas M'_i) para cada puerto visitado. La verificación del certificado consiste en:

1. **Verificación de ruta:** Comprobar que π es una ruta simple (excepto por v_0 y v_{m+1}) y que el tiempo total de navegación cumple $\sum d_{v_i, v_{i+1}} \leq T_{max}$. Esta operación se realiza en $O(m)$ mediante un vector de presencia para los nodos.
2. **Verificación de carga:** En cada puerto v_i , se calcula el peso total sumando las mercancías retenidas en la bodega L_i después de vender y las nuevas adquisiciones $C_i \subseteq M_i$, verificando que $\sum_{G=(i_g, k, w_g, p_g^{compra}) \in L_i} w_g + \sum_{(w, p^{compra}, p^{venta}) \in C_i} w \leq C_{max}$. La complejidad es $O(m \cdot \max |L_i|)$, lo cual es polinomial respecto a la cantidad de mercancías.
3. **Verificación financiera:** Simular el flujo de caja K_i sumando ingresos por ventas M'_i y restando costos de compra C_i . Se verifica que en todo momento $K_i \geq k_{\min}$ y que al regresar a Ámsterdam el capital final $K_f \geq K_{objetivo}$.

Como todas las operaciones de verificación se realizan en tiempo polinomial respecto al tamaño de la entrada, el problema **pertenece a NP**.

3.2. NP-Compleitud mediante Reducción de TSP

Para demostrar la NP-Compleitud, se reduce una instancia del problema de decisión de **TSP** (NP-Completo) a una instancia del problema.

Definición de TSP: Dado un grafo $G_{TSP} = (V', E', \omega)$ y una distancia límite L , ¿existe un tour que visite todos los nodos de V' tal que la distancia recorrida sea a lo sumo L ?

$\omega(e_{i,j})$: distancia entre nodos i y j .

3.2.1. Construcción de la reducción:

Dada una instancia de **TSP**, se construye una instancia del problema de la siguiente forma:

- **Grafo y Tiempos:**

$$T_{max} = L \quad O(1),$$

$$V = V' \cup \{s\} \quad O(n) \quad \text{y}$$

$$d_{i,j} = \begin{cases} \text{peso}(e_{i,j}) & \text{si } e_{i,j} \in E' \wedge i, j \neq s, \\ 0 & i = s \vee j = s, \\ \infty & \text{si } e_{i,j} \notin E'. \end{cases} \quad O(n^2)$$

- **Incentivo a la Visita:** En cada puerto $i \in V \setminus \{s\}$, se define una única mercancía $m'_i \in M_i$ con $w = 0$ y $p_{compra} = 0$. Su precio de venta es 0 en todo puerto, excepto en Ámsterdam (s), donde $p_{venta} = 1$. $O(n)$
- **Condiciones Iniciales:** $k_0 = 0$, $k_{\min} = 0$, $C_{max} = \infty$. $O(1)$
- **Meta de Ganancia:** Se define la meta de capital final como $K_{objetivo} = |V| - 1$. $O(1)$

Por tanto la reducción es ejecutable en **tiempo polinomial**.

3.2.2. Demostración de equivalencia:

- $(\Rightarrow)$ Si **TSP** tiene un tour de longitud de a lo sumo L , el capitán puede seguir dicho tour visitando cada puerto una vez, recoger las $|V| - 1$ mercancías y volver a Ámsterdam dentro del tiempo T_{max} . Al venderlas, obtiene $K_f = |V| - 1$, satisfaciendo el problema.
- $(\Leftarrow)$ Si el problema tiene solución con capital de al menos $K_{objetivo}$, el capitán necesariamente visitó todos los puertos para recolectar el capital (dada la ausencia de otros beneficios). Como lo hizo en un tiempo menor de T_{max} , el recorrido representa un tour válido para **TSP**.

Dado que **TSP** se reduce al problema, se concluye que es **NP-Completo**.

4. Diseño de soluciones algorítmicas

Para resolver el problema, se han diseñado dos soluciones por fuerza bruta y una solución eficiente que combina un algoritmo voraz (*greedy*) con meta-heurística (*simulate annealing*). Todas tienen las mismas entradas, emplean las mismas estructuras de datos y producen una salida.

4.1. Entradas

- n : número de puertos.
- $d[i][j]$: matriz de distancias entre puertos ($d_{i,j}$).
- $T_{\text{máx}}$: tiempo máximo permitido para viajar.
- $C_{\text{máx}}$: capacidad máxima de carga del barco.
- k_0 : capital inicial disponible.
- $k_{\text{mín}}$: capital mínimo requerido en todo momento.
- M_i : conjunto de ítems disponibles en el puerto i , cada uno con:
 - $w_{i,k}$: peso del ítem k en el puerto i .
 - $p_{i,k}^{\text{buy}}$: precio de compra del ítem.
 - $p_{i,k}^{\text{sell}}$: precio de venta del ítem.

4.2. Salida

- Ganancia máxima obtenida tras visitar los puertos, comprar y vender ítems, respetando las restricciones de tiempo y capacidad.

4.3. Estructuras de datos utilizadas

- **Item**: representa un ítem disponible en un puerto. Contiene:
 - w : peso del ítem ($w_{i,k}$).
 - buy_price : precio de compra ($p_{i,k}^{\text{buy}}$).
 - sell_price : precio de venta ($p_{i,k}^{\text{sell}}$).
- **Merchandise**: representa una mercancía que ya ha sido comprada y está a bordo. Contiene:
 - i : puerto en el que fue adquirida.
 - k : índice del ítem.
 - w : peso de la mercancía.
 - buy_price : precio de compra pagado.
- **itemsByPort**: matriz de dimensión $n \times m$ que contiene los ítems disponibles en cada puerto.

- n : número de puertos.
- m : número de ítems diferentes (se asume igual en todos los puertos por comodidad).
- La ausencia de un ítem se representa como un objeto **Item** con:
 - peso $w = \infty$,
 - precio de compra $p^{buy} = \infty$,
 - precio de venta $p^{sell} = -\infty$.
- **State**: representa el estado completo del comerciante en un momento dado de la simulación. Contiene:
 - *current_port*: puerto actual en el que se encuentra.
 - *capital*: capital disponible en ese instante.
 - *time*: tiempo acumulado de viaje.
 - *used_capacity*: capacidad de carga utilizada en el barco.
 - *route*: lista de puertos visitados en orden.
 - *inventory*: lista de mercancías (**Merchandise**) que están actualmente a bordo.

4.4. Descripción de las soluciones

El Algoritmo 1 es una de las implementaciones de fuerza bruta, que se describe a continuación.

Descripción del flujo del algoritmo. El algoritmo comienza en la función **Solve**, que inicializa el estado de los puertos visitados y llama a la función **Buy** para explorar las decisiones iniciales de compra.

La función **Buy** evalúa, en cada puerto, si es posible adquirir una mercancía dadas las restricciones de capital mínimo y capacidad de carga. Si la compra es viable, la mercancía se añade temporalmente a la carga y se realiza una llamada recursiva para continuar explorando. También se contempla la opción de no comprar y avanzar al siguiente ítem. En todo momento se compara la ganancia obtenida por comprar con la ganancia de viajar directamente (**Travel**).

La función **Sell** se encarga de explorar las posibles ventas de las mercancías que están a bordo en un puerto dado. Para cada mercancía se considera la opción de venderla o mantenerla, actualizando el capital y la capacidad, y llamando recursivamente a sí misma para evaluar el resto de mercancías. El resultado es la mejor ganancia entre vender y no vender.

La función **Travel** evalúa la posibilidad de desplazarse a otros puertos no visitados, siempre que el tiempo restante lo permita. Marca el puerto como visitado, llama a **Sell** para explorar las transacciones en ese puerto, y luego desmarca el puerto para permitir otras rutas. El valor devuelto es la máxima ganancia alcanzable considerando todas las rutas posibles.

En conjunto, el flujo del algoritmo es una búsqueda exhaustiva recursiva que combina decisiones de compra, venta y viaje, manteniendo en todo momento las restricciones de tiempo, capacidad y capital mínimo.

Correctitud. El algoritmo es correcto porque explora todas las combinaciones posibles de decisiones (comprar, vender, viajar o no hacerlo) y siempre mantiene las restricciones globales. Al devolver el máximo entre las ganancias obtenidas en cada rama de decisión, garantiza que el resultado final corresponde a la mejor estrategia posible. La recursividad asegura que no se omite ninguna ruta ni combinación de transacciones.

Complejidad Temporal El algoritmo explora todas las posibles rutas que comienzan en Ámsterdam y regresan dentro del tiempo máximo permitido. La clave está en que no basta con elegir un subconjunto de puertos: también importa el **orden en que se visitan**. Por eso, lo que se explora son **variaciones sin repetición**: todas las secuencias posibles de longitud k tomadas de n puertos. El número de secuencias de longitud k es:

$$P(n, k) = \frac{n!}{(n - k)!}.$$

Dado que el algoritmo permite rutas de cualquier longitud k (desde 0 hasta n), el total de rutas posibles es:

$$\sum_{k=0}^n P(n, k).$$

Esta sumatoria es del orden de $n!$ (tiende a $e \cdot n!$). En general:

- Exploración de rutas: $\Theta(n!)$, por el orden de visita de los puertos.
- Decisiones de compra/venta: $\Theta(2^m)$ por puerto, ya que cada ítem puede comprarse/venderse o no.
- Complejidad Total:

$$O(n! \cdot 2^m).$$

lo que lo hace impracticable para instancias grandes. Sin embargo, es útil como referencia exacta para instancias pequeñas y como base para comparar heurísticas.

El Algoritmo 2 es otra implementación de fuerza bruta, que se describe a continuación.

Descripción del flujo del algoritmo. El algoritmo comienza en la función **Solve**, que prepara las estructuras iniciales y llama a la función interna **_Solve**. Esta función recursiva construye rutas posibles desde Ámsterdam, explorando todos los puertos alcanzables dentro del tiempo máximo $T_{\text{máx}}$. Cada vez que se regresa al puerto inicial, se evalúa la ruta completa mediante la función **_ProfitByRoute**.

La función **_ProfitByRoute** calcula la ganancia máxima posible siguiendo la secuencia de puertos de la ruta. En cada puerto se consideran todas las combinaciones de ventas (**_Sell**) de las mercancías a bordo y todas las combinaciones de compras (**_Buy**) de las mercancías disponibles en el puerto. Después de aplicar las decisiones de compra y venta, se actualizan las cargas y se continúa recursivamente hacia el siguiente puerto de la ruta. La función **_SellAll** se utiliza para liquidar todas las mercancías al final del viaje, y **_CalculateWeight** asegura que las restricciones de capacidad se respeten.

En conjunto, el flujo del algoritmo es una búsqueda exhaustiva que combina la exploración de rutas posibles con la evaluación de todas las combinaciones de compra y venta en cada puerto, manteniendo las restricciones de tiempo, capacidad y capital mínimo.

Correctitud. El algoritmo es correcto porque explora todas las rutas factibles que comienzan y terminan en Ámsterdam dentro del tiempo máximo, y para cada ruta considera exhaustivamente todas las combinaciones de compra y venta de mercancías. Al devolver siempre el máximo entre las ganancias obtenidas en cada rama de decisión, garantiza que el resultado final corresponde a la mejor estrategia posible. La recursividad asegura que no se omite ninguna ruta ni ninguna combinación de transacciones.

Complejidad Temporal. El algoritmo explora todas las posibles rutas de longitud variable que comienzan en Ámsterdam y regresan dentro del tiempo máximo permitido. No basta con elegir un subconjunto de puertos: también importa el **orden en que se visitan**. Por eso, lo que se explora son **variaciones sin repetición**: todas las secuencias posibles de longitud k tomadas de n puertos. El número de secuencias de longitud k es:

$$P(n, k) = \frac{n!}{(n - k)!}.$$

Dado que el algoritmo permite rutas de cualquier longitud k (desde 0 hasta n), el total de rutas posibles es:

$$\sum_{k=0}^n P(n, k).$$

Esta sumatoria es del orden de $n!$ (tiende a $e \cdot n!$).

En cuanto a las decisiones de compra y venta, en cada puerto se exploran todos los subconjuntos de mercancías a bordo para vender ($2^{|L|}$ posibilidades) y todos los subconjuntos de mercancías disponibles para comprar (2^m posibilidades). $|L| \leq m$, por lo que el espacio de decisiones por puerto es $O(4^m)$.

- Exploración de rutas: $\Theta(n!)$, por el orden de visita de los puertos.
- Decisiones de compra/venta: $O(4^m)$ por puerto, considerando todas las combinaciones de venta y compra.
- Complejidad Total:

$$O(n! \cdot 4^m).$$

Esto lo hace impracticable para instancias grandes, pero útil como referencia exacta para instancias pequeñas y como base para comparar heurísticas.

El Algoritmo 3 es una implementación aproximada que combina una solución inicial *greedy* con el método de *Simulated Annealing*, y se describe a continuación.

Descripción del flujo del algoritmo. El algoritmo comienza en la función **Solve**, que genera una solución inicial mediante la función **GreedyInitialSolution**. Esta solución se construye recorriendo los puertos de manera heurística, vendiendo y comprando mercancías según convenga, y asegurando que se pueda regresar a Ámsterdam dentro del tiempo máximo permitido.

La función **GreedyInitialSolution** utiliza tres funciones auxiliares:

- **SellInCurrentPort**: vende las mercancías en el puerto actual si el precio de venta es mayor que el de compra.

- **BuyInCurrentPort**: selecciona mercancías prometedoras en el puerto actual usando una heurística tipo mochila 0/1, priorizando la relación ganancia/peso.
- **SelectNextPortGreedy**: elige el próximo puerto a visitar en función de una puntuación heurística que combina distancia y cantidad de ítems disponibles.

Al finalizar la ruta *greedy*, la función **SellAllInAmsterdam** liquida todo el inventario en el puerto inicial para calcular el capital final de la solución inicial.

Una vez obtenida la solución inicial, la función **SimulatedAnnealing** se encarga de mejorarla. Este procedimiento mantiene un estado actual y un estado óptimo encontrado hasta el momento. En cada iteración:

- Se genera una ruta vecina mediante **GenerateNeighbor**, que modifica la ruta actual aplicando operaciones como intercambio, inserción, eliminación o inversión de segmentos.
- La ruta vecina se evalúa con **SimulateRoute**, que simula el recorrido completo aplicando las funciones de compra y venta en cada puerto y verificando las restricciones de capacidad y capital.
- El capital final de la ruta simulada se calcula con **EvaluateState**, que vende todo el inventario en Ámsterdam.
- El criterio de aceptación de **SimulatedAnnealing** permite aceptar soluciones peores con cierta probabilidad dependiente de la temperatura, lo que ayuda a escapar de óptimos locales.

El proceso de enfriamiento reduce gradualmente la temperatura, haciendo que el algoritmo se vuelva más estricto en la aceptación de soluciones. Al finalizar las iteraciones, se devuelve el mejor estado encontrado.

En conjunto, el flujo del algoritmo combina una construcción inicial *greedy* con una búsqueda aproximada basada en *Simulated Annealing*, explorando rutas vecinas y evaluando las decisiones de compra y venta de mercancías para aproximarse a una solución de alta calidad.

5. Análisis empírico

Con el objetivo de evaluar el desempeño del algoritmo eficiente frente al enfoque de fuerza bruta, se realizaron una serie de experimentos variando tanto el número de ítems como el número de puertos en las instancias del problema. El análisis completo se encuentra en el directorio `../src/validation`.

5.1. Calidad de la solución

Al comparar la calidad de la solución obtenida por el algoritmo eficiente frente a la solución óptima de fuerza bruta tanto para variaciones de puertos (ver Figura 1) como variaciones de items (ver Figura 2) la solución eficiente siempre se mantiene sobre el 70 % de la solución óptima.

5.2. Tiempos de ejecución

El algoritmo de fuerza bruta presenta tiempos prácticamente nulos para instancias muy pequeñas, pero crecen rápidamente conforme aumenta el tamaño. Mientras que el algoritmo eficiente mantiene tiempos muy bajos y estables en todas las instancias. Este comportamiento se observa en las comparaciones de ambos algoritmos, en las Figura 3 y Figura 4

5.3. Tamaño máximo de instancia para fuerza bruta

El algoritmo de fuerza bruta puede resolver en tiempos razonables (menos de un minuto ver Figura 5) instancias de 3 puertos y hasta 10 items. A partir de 6 puertos con 4 items, los tiempos crecen de forma exponencial (comportamiento esperado por la complejidad) y superan los 60 segundos (ver Figura 6).

5.4. Comportamiento del algoritmo eficiente

Para las instancias del problema donde la solución óptima es no viajar o hacer ventas en puertos inmediatos a los puertos donde se realizó la compra el algoritmo eficiente devuelve una solución cercana al óptimo. Sin embargo, para aquellas instancias donde la mejor estrategia para una mercancía es venderla en un puerto muy distante a donde se compró (con respecto a la cantidad de puertos que los separan en la ruta), funciona peor porque se basa en no retener mercancías con ganancia conocida.

En instancias pequeñas casi siempre alcanza el óptimo, dado que el espacio de soluciones es pequeño y potencialmente lo explora en casi su totalidad.

Anexos

Algorithm 1 Fuerza Bruta. Joel Aparicio Tamayo

```
1: Function Solve( $n, d, T_{\text{máx}}, C_{\text{máx}}, k_0, k_{\text{mín}}, \text{itemsByPort}$ )
2: Initialize  $\text{portsVisited} \leftarrow$  list of False
3:  $\text{portsVisited}[0] \leftarrow$  True
4:  $m \leftarrow$  number of items at port 0
5: return  $\text{máx}(k_0, \text{Buy}(n, d, T_{\text{máx}}, C_{\text{máx}}, k_0, k_{\text{mín}}, m,$ 
6:    $\text{itemsByPort}, 0, [ ], \text{portsVisited}, 0))$ 
7:
8: Function Sell( $n, d, T_{\text{máx}}, C_{\text{máx}}, k_0, k_{\text{mín}}, \text{size},$ 
9:    $\text{itemsByPort}, \text{port}, \text{itemsOnBoard}, \text{portsVisited}, j$ )
10:  $\text{purchaseGain} \leftarrow \text{Buy}(\dots)$ 
11: if  $j \geq$  length of  $\text{itemsOnBoard}$  then
12:   return  $\text{purchaseGain}$ 
13: end if
14: if  $\text{itemsOnBoard}$  not empty then
15:   Remove item  $m$  at position  $j$ 
16:    $\text{sellPrice} \leftarrow \text{itemsByPort}[\text{port}][m.k].\text{sellPrice}$ 
17:    $\text{sellGain} \leftarrow \text{Sell}(\dots)$  with updated capital and capacity
18:   Reinsert item  $m$ 
19:    $\text{sellGain} \leftarrow \text{máx}(\text{sellGain}, \text{Sell}(\dots, j + 1))$ 
20: end if
21: return  $\text{máx}(\text{purchaseGain}, \text{sellGain})$ 
22:
23: Function Buy( $n, d, T_{\text{máx}}, C_{\text{máx}}, k_0, k_{\text{mín}}, \text{size},$ 
24:    $\text{itemsByPort}, \text{port}, \text{itemsOnBoard}, \text{portsVisited}, j$ )
25:  $\text{travelGain} \leftarrow k_0$ 
26: if  $k_{\text{mín}} > \text{travelGain}$  then
27:   return  $-1$ 
28: end if
29:  $\text{travelGain} \leftarrow \text{Travel}(\dots)$ 
30: if  $j \geq \text{size}$  then
31:   return  $\text{travelGain}$ 
32: end if
33: if  $k_{\text{mín}} \leq \text{funds}$  and  $\text{capacityLeft} \geq 0$  then
34:   Append to  $\text{itemsOnBoard}$ : Merchandise( $i = \text{port}, k = j,$ 
35:      $w = \text{currentItem}.w, \text{buyPrice} = \text{currentItem}.buyPrice)$ 
36:    $\text{purchase} \leftarrow \text{Buy}(\dots, j + 1)$  with updated capital and capacity
```

```

37:   Remove last item from itemsOnBoard
38: end if
39: purchase  $\leftarrow \max(\text{purchase}, \text{Buy}(\dots, j + 1))$  with current capital and ca-
    pacity
40: return  $\max(\text{purchase}, \text{travelGain})$ 
41:
42: Function Travel(n, d, Tmáx, Cmáx, k0, kmín, size,
43:   itemsByPort, port, itemsOnBoard, portsVisited)
44: finalGain  $\leftarrow k_0 + \sum$  selling prices at initial port
45: maxGain  $\leftarrow -1$ 
46: for each port i do
47:   if not visited and enough time then
48:     Mark port i as visited
49:     nextGain  $\leftarrow \text{Sell}(\dots)$  at port i
50:     maxGain  $\leftarrow \max(\text{maxGain}, \text{nextGain})$ 
51:     Unmark port i
52:   end if
53: end for
54: return maxGain

```

Algorithm 2 Fuerza Bruta. Claudia Hernández Pérez

```

1: Function Solve(n, d, Tmáx, Cmáx, k0, kmín, itemsByPort)
2: return  $\text{Solve}(n, d, T_{\text{máx}}, C_{\text{máx}}, k_0, k_{\text{mín}}, \text{itemsByPort},$ 
3:    $0, k_0, [\text{False for } i = 1..n], [ ], [[ ] \text{ for } i = 1..n])$ 
4:
5: Function  $\text{Solve}(n, d, T_{\text{máx}}, C_{\text{máx}}, k_0, k_{\text{mín}}, \text{itemsByPort},$ 
6:    $\text{port}_i, \text{maxProfit}, \text{visited}, \text{route}, L_i)$ 
7: if  $\text{port}_i = 0$  and  $\text{visited}[0] = \text{True}$  then
8:   Append 0 to route
9:   maxProfit  $\leftarrow \max(\text{maxProfit},$ 
10:     $\text{ProfitByRoute}(C_{\text{máx}}, k_0, k_{\text{mín}}, \text{itemsByPort},$ 
11:     $\text{route}, L_i, 0, \text{maxProfit}))$ 
12:   Remove last element from route
13:   return maxProfit
14: end if
15: Append porti to route
16: for each port j in  $0..n - 1$  do
17:   if  $\text{port}_i \neq \text{port}_j$  and not  $\text{visited}[j]$  and

```



```

18:          $T_{\max} - (d[port_i][j] + d[j][0]) \geq 0$  then
19:      $visited[j] \leftarrow \text{True}$ 
20:      $maxProfit \leftarrow \max(maxProfit,$ 
21:          $\text{Solve}(n, d, T_{\max} - d[port_i][j], C_{\max}, k_0,$ 
22:              $k_{\min}, itemsByPort, j, maxProfit, visited, route, L_i))$ 
23:      $visited[j] \leftarrow \text{False}$ 
24: end if
25: end for
26: Remove last element from route
27: return  $maxProfit$ 
28:
29: Function  $\text{ProfitByRoute}(C_{\max}, k_0, k_{\min}, itemsByPort,$ 
30:      $route, L_i, i, maxProfit)$ 
31: if  $k_0 < 0$  then
32:     return  $-\infty$ 
33: end if
34:  $port \leftarrow route[i]$ 
35: if  $i = |route| - 1$  then
36:     return  $k_0 + \text{SellAll}(L_i[port], itemsByPort[port])$ 
37: end if
38: for each sell in  $\text{Sell}(L_i[port], itemsByPort[port], [], 0, [])$  do
39:      $rest \leftarrow L_i[port] \setminus sell[0]$ 
40:      $sold \leftarrow sell[0]$ 
41:      $capacitySold \leftarrow \text{CalculateWeight}(sold)$ 
42:     for each buy in  $\text{Buy}(port, itemsByPort[port], [], 0, [],$ 
43:          $C_{\max} + capacitySold, k_0 + sell[1] - k_{\min})$  do
44:          $capacityBought \leftarrow \text{CalculateWeight}(buy[0])$ 
45:          $L_i[route[i + 1]] \leftarrow rest + buy[0]$ 
46:          $maxProfit \leftarrow \max(maxProfit,$ 
47:              $\text{ProfitByRoute}(C_{\max} + capacitySold - capacityBought,$ 
48:                  $k_0 + sell[1] - k_{\min} - buy[1], k_{\min},$ 
49:                  $itemsByPort, route, L_i, i + 1, maxProfit))$ 
50:     end for
51: end for
52: return  $maxProfit$ 
53:
54: Function  $\text{Sell}(goods, itemsAtPort, goodsSold, i, result)$ 
55: if  $i = |goods|$  then
56:      $sold \leftarrow \sum_{j \in goodsSold} itemsAtPort[j.k].sellPrice$ 
57:     Append ( $goodsSold, sold$ ) to result
58: return result

```

```

59: end if
60:  $merchandise \leftarrow goods[i]$ 
61: Append  $merchandise$  to  $goodsSold$ 
62:  $\_Sell(goods, itemsAtPort, goodsSold, i + 1, result)$ 
63: Remove last element from  $goodsSold$ 
64:  $\_Sell(goods, itemsAtPort, goodsSold, i + 1, result)$ 
65: return  $result$ 
66:
67: Function  $\_SellAll(goods, itemsAtPort)$ 
68: return  $\sum_{j \in goods} itemsAtPort[j.k].sellPrice$ 
69:
70: Function  $\_Buy(port, itemsAtPort, goodsSold, i, result, C_{\max}, K)$ 
71: if  $i = |itemsAtPort|$  then
72:    $cost \leftarrow \sum_{j \in goodsSold} j.buyPrice$ 
73:   Append  $(goodsSold, cost)$  to  $result$ 
74:   return  $result$ 
75: end if
76:  $merchandise \leftarrow Merchandise(port, i, itemsAtPort[i].w,$ 
77:    $itemsAtPort[i].buyPrice)$ 
78: if  $C_{\max} - merchandise.w \geq 0$  and  $K - merchandise.buyPrice \geq 0$  then
79:   Append  $merchandise$  to  $goodsSold$ 
80:    $\_Buy(port, itemsAtPort, goodsSold, i + 1, result,$ 
81:      $C_{\max} - merchandise.w, K - merchandise.buyPrice)$ 
82:   Remove last element from  $goodsSold$ 
83: end if
84:  $\_Buy(port, itemsAtPort, goodsSold, i + 1, result, C_{\max}, K)$ 
85: return  $result$ 
86:
87: Function  $\_CalculateWeight(goods)$ 
88: return  $\sum_{item \in goods} item.w$ 

```

Algorithm 3 Solución Eficiente

```

1: Function  $Solve(n, d, T_{\max}, C_{\max}, k_0, k_{\min}, itemsByPort)$ 
2:  $initialState \leftarrow GreedyInitialSolution()$ 
3: if  $|initialState.route| = 1$  then
4:   return  $initialState.capital$ 
5: end if
6:  $bestState \leftarrow SimulatedAnnealing(initialState, maxIter = 2000)$ 

```

```

7:  $finalCapital \leftarrow \text{EvaluateState}(bestState, itemsByPort)$ 
8: return  $\text{máx}(finalCapital, k_0)$ 
9:
10: Function GreedyInitialSolution()
11:  $state \leftarrow \text{State}(current\_port = 0, capital = k_0, time = 0,$ 
12:    $usedCapacity = 0, route = [0], inventory = [ ])$ 
13:  $unvisited \leftarrow \{1..n - 1\}$ 
14: while  $unvisited \neq \emptyset$  do
15:    $state \leftarrow \text{SellInCurrentPort}(state, itemsByPort)$ 
16:    $nextPort \leftarrow \text{SelectNextPortGreedy}(state, unvisited, d, T_{\text{máx}})$ 
17:   if  $nextPort = None$  then
18:     break
19:   end if
20:    $state \leftarrow \text{BuyInCurrentPort}(state, itemsByPort, nextPort)$ 
21:   if  $state.capital < k_{\text{mín}}$  then
22:     break
23:   end if
24:    $state.capital \leftarrow state.capital - k_{\text{mín}}$ 
25:    $state.time \leftarrow state.time + d[state.current\_port][nextPort]$ 
26:    $state.current\_port \leftarrow nextPort$ 
27:   Append  $nextPort$  to  $state.route$ 
28:   Remove  $nextPort$  from  $unvisited$ 
29: end while
30: if  $state.current\_port \neq 0$  then
31:    $state.time \leftarrow state.time + d[state.current\_port][0]$ 
32:    $state.capital \leftarrow state.capital - k_{\text{mín}}$ 
33:    $state.current\_port \leftarrow 0$ 
34:   Append 0 to  $state.route$ 
35:    $state \leftarrow \text{SellAllInAmsterdam}(state, itemsByPort)$ 
36: end if
37: return  $state$ 
38:
39: Function SellInCurrentPort( $state, itemsByPort$ )
40:  $newState \leftarrow state.copy()$ 
41:  $newInventory \leftarrow [ ]$ 
42: for each  $merch$  in  $state.inventory$  do
43:    $(i, k, w, buyPrice) \leftarrow merch$ 
44:   if  $i \neq state.current\_port$  then
45:      $sellPrice \leftarrow itemsByPort[state.current\_port][k].sellPrice$ 
46:     if  $sellPrice > buyPrice$  then
47:        $newState.capital \leftarrow newState.capital + sellPrice$ 

```

```

48:     else
49:         Append merch to newInventory
50:     end if
51: else
52:     Append merch to newInventory
53: end if
54: end for
55: newState.inventory  $\leftarrow$  newInventory
56: return newState
57:
58: Function BuyInCurrentPort(state, itemsByPort, nextPort)
59: newState  $\leftarrow$  state.copy()
60: availableCapacity  $\leftarrow$   $C_{\text{máx}}$  - state.usedCapacity
61: availableCapital  $\leftarrow$  state.capital -  $k_{\text{mín}}$ 
62: candidates  $\leftarrow$  [ ]
63: for each item k in itemsByPort[state.current_port] do
64:     if item.w  $\leq$  availableCapacity and item.buyPrice  $\leq$  availableCapital
        then
65:         profit  $\leftarrow$  itemsByPort[nextPort][k].sellPrice - item.buyPrice
66:         if profit > 0 then
67:             Append (k, item.w, item.buyPrice, profit) to candidates
68:         end if
69:     end if
70: end for
71: Sort candidates by profit/weight descending
72: for each (k, w, buyPrice, profit) in candidates do
73:     if availableCapacity  $\geq$  w and availableCapital  $\geq$  buyPrice then
74:         newState.capital  $\leftarrow$  newState.capital - buyPrice
75:         newState.usedCapacity  $\leftarrow$  newState.usedCapacity + w
76:         Append Merchandise(port, k, w, buyPrice) to newState.inventory
77:         availableCapacity  $\leftarrow$  availableCapacity - w
78:         availableCapital  $\leftarrow$  availableCapital - buyPrice
79:     end if
80: end for
81: return newState
82:
83: Function SelectNextPortGreedy(state, unvisited, d,  $T_{\text{máx}}$ )
84: bestPort  $\leftarrow$  None
85: bestScore  $\leftarrow$   $-\infty$ 
86: current  $\leftarrow$  state.current_port
87: for each port in unvisited do

```

```

88:   $travelTime \leftarrow d[current][port]$ 
89:   $timeBack \leftarrow d[port][0]$ 
90:   $totalTime \leftarrow state.time + travelTime + timeBack$ 
91:  if  $totalTime \leq T_{\max}$  then
92:     $distanceScore \leftarrow 1/(travelTime + 10^{-6})$ 
93:     $itemsCount \leftarrow |itemsByPort[port]|$ 
94:     $itemScore \leftarrow \min(itemsCount/10, 1)$ 
95:     $score \leftarrow distanceScore + itemScore$ 
96:    if  $score > bestScore$  then
97:       $bestScore \leftarrow score$ 
98:       $bestPort \leftarrow port$ 
99:    end if
100:  end if
101: end for
102: return  $bestPort$ 
103:
104: Function SellAllInAmsterdam( $state, itemsByPort$ )
105:  $newState \leftarrow state.copy()$ 
106: for each  $merch$  in  $state.inventory$  do
107:    $(i, k, w, buyPrice) \leftarrow merch$ 
108:    $sellPrice \leftarrow itemsByPort[0][k].sellPrice$ 
109:    $newState.capital \leftarrow newState.capital + sellPrice$ 
110: end for
111:  $newState.inventory \leftarrow []$ 
112: return  $newState$ 
113:
114: Function EvaluateState( $state, itemsByPort$ )
115:  $newState \leftarrow SellAllInAmsterdam(state, itemsByPort)$ 
116: return  $newState.capital$ 
117:
118: Function SimulateRoute( $route, d, itemsByPort$ )
119: if  $route = []$  or  $route[0] \neq 0$  then
120:   return None
121: end if
122:  $state \leftarrow State(current\_port = 0, capital = k_0, time = 0,$ 
123:    $usedCapacity = 0, route = [0], inventory = [])$ 
124: for  $i = 1$  to  $|route| - 1$  do
125:    $state \leftarrow SellInCurrentPort(state, itemsByPort)$ 
126:    $nextPort \leftarrow route[i]$ 
127:    $travelTime \leftarrow d[state.current\_port][nextPort]$ 
128:   if  $state.time + travelTime > T_{\max}$  then

```

```

129:     return None
130: end if
131:  $state \leftarrow \text{BuyInCurrentPort}(state, itemsByPort, nextPort)$ 
132: if  $state.capital < 0$  or  $state.usedCapacity > C_{\text{máx}}$  then
133:     return None
134: end if
135:  $state.time \leftarrow state.time + travelTime$ 
136:  $state.current\_port \leftarrow nextPort$ 
137:  $state.capital \leftarrow state.capital - k_{\text{mín}}$ 
138: Append  $nextPort$  to  $state.route$ 
139: end for
140:  $state \leftarrow \text{SellAllInAmsterdam}(state, itemsByPort)$ 
141: return  $state$ 
142:
143: Function GenerateNeighbor( $state$ )
144:  $route \leftarrow state.route.copy()$ 
145: if  $|route| = 3$  then
146:      $available \leftarrow \{1..n - 1\} \setminus route$ 
147:     if  $available \neq \emptyset$  then
148:          $insertPos \leftarrow$  random position in  $[1, |route| - 2]$ 
149:          $insertPort \leftarrow$  random choice from  $available$ 
150:         Insert  $insertPort$  at  $insertPos$  in  $route$ 
151:     end if
152:     return  $route$ 
153: end if
154:  $operation \leftarrow$  random choice in  $\{\text{swap}, \text{insert}, \text{remove}, \text{reverse}\}$ 
155: if  $operation = \text{swap}$  then
156:     Swap two random ports in  $route$  (not Amsterdam)
157: else if  $operation = \text{insert}$  then
158:     Insert a random unvisited port into  $route$  (excluding first and last
        positions)
159: else if  $operation = \text{remove}$  then
160:     Remove a random port from  $route$  (not Amsterdam)
161: else if  $operation = \text{reverse}$  then
162:     Reverse a random segment of  $route$  (excluding first and last positions)
163: end if
164: return  $route$ 
165:
166: Function SimulatedAnnealing( $initialState, maxIter$ )
167:  $currentState \leftarrow initialState$ 
168:  $bestState \leftarrow initialState.copy()$ 

```

```

169: currentValue  $\leftarrow$  EvaluateState(currentState, itemsByPort)
170: bestValue  $\leftarrow$  currentValue
171: temperature  $\leftarrow$  1000
172: coolingRate  $\leftarrow$  0,995
173: for iter = 1 to maxIter do
174:   newRoute  $\leftarrow$  GenerateNeighbor(currentState)
175:   newState  $\leftarrow$  SimulateRoute(newRoute, d, itemsByPort)
176:   if newState = None then
177:     continue
178:   end if
179:   newValue  $\leftarrow$  EvaluateState(newState, itemsByPort)
180:    $\Delta \leftarrow$  newValue - currentValue
181:   if  $\Delta > 0$  or random()  $< e^{\Delta/\text{temperature}}$  then
182:     currentState  $\leftarrow$  newState
183:     currentValue  $\leftarrow$  newValue
184:     if currentValue  $>$  bestValue then
185:       bestState  $\leftarrow$  currentState.copy()
186:       bestValue  $\leftarrow$  currentValue
187:     end if
188:   end if
189:   temperature  $\leftarrow$  temperature  $\cdot$  coolingRate
190: end for
191: return bestState

```

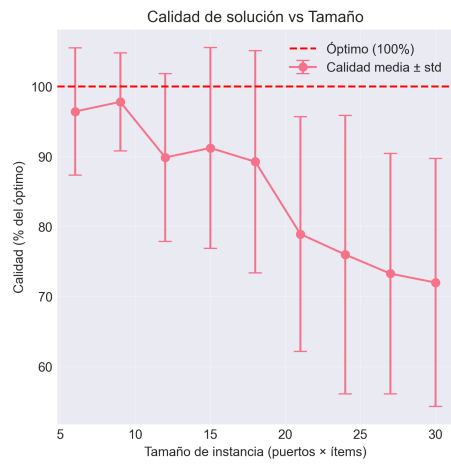


Figura 1: Calidad con puertos fijos

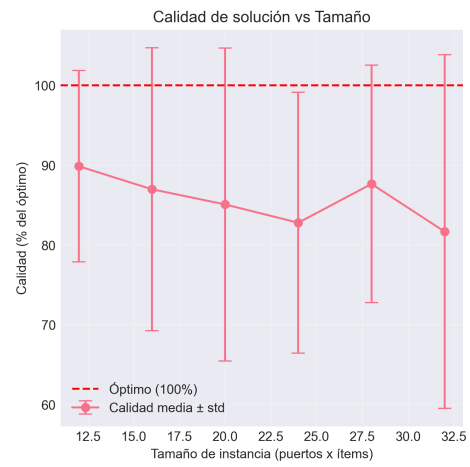


Figura 2: Calidad con items fijos

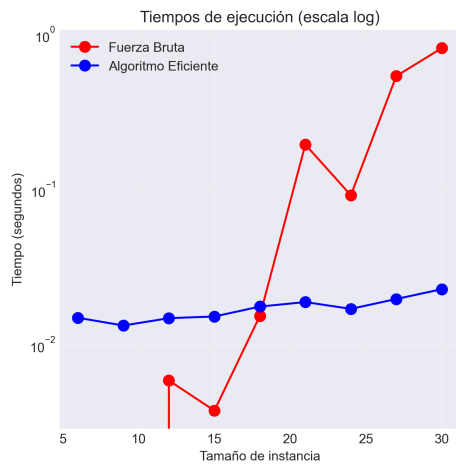


Figura 3: Tiempos con puertos fijos

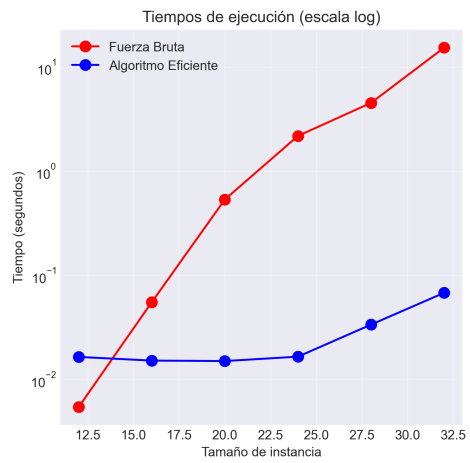


Figura 4: Tiempo con items fijos

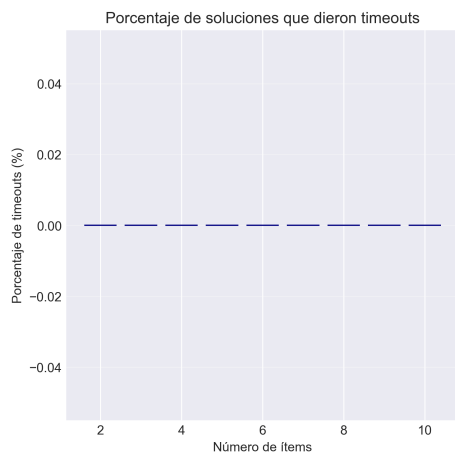


Figura 5: Timeouts con puertos fijos

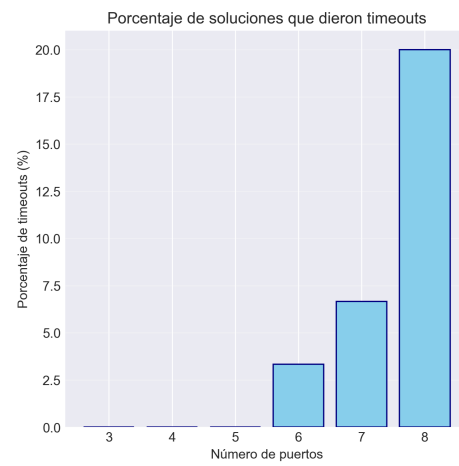


Figura 6: Timeouts con ítems fijos